

Kairos Edge Deployment on BCM — POC

Executive Summary

We propose deploying Spectro Cloud's Kairos OS to bare-metal compute nodes using your existing Bright Cluster Manager (BCM) infrastructure. The solution leverages BCM's native PXE, TFTP, and provisioning pipeline, adding a Kairos-specific software image that writes an immutable Kairos root filesystem via `dd` directly from BCM's HTTP server.

The entire deployment is automated with Ansible. A one-time Kairos image build takes approximately 30 minutes; each subsequent compute node is provisioned in 10–15 minutes. Re-registration of previously-provisioned edge hosts is fully automated: no manual Palette UI steps are required on re-deploys.

Why This Approach

The Problem

Kairos and BCM use fundamentally different installation processes:

- **BCM's provisioning** writes a traditional Linux root filesystem to the node's disk via rsync, backed by NFS and a systemd-managed chroot.
- **Kairos requires** a specific GPT partition layout (`COS_GRUB` , `COS_OEM` , `COS_RECOVERY` , `COS_STATE` , `COS_PERSISTENT`), an EFI System Partition with a GRUB-EFI loader, and an immutable squashfs root booted via `cos-img`.

The two approaches are incompatible out of the box. Running `kairos-agent install` inside BCM's installer environment isn't viable — `kairos-agent` expects to boot from its own ISO and partition disks on its own terms.

Typical alternatives all have serious downsides:

- **Manual USB/ISO installation per node** — doesn't scale, requires physical presence.
- **Custom PXE infrastructure separate from BCM** — duplicates what BCM already provides, requires separate DHCP/TFTP, creates network conflicts with existing BCM provisioning.
- **Replacing BCM entirely** — loses HPC cluster management capabilities.

Our Solution

We inject Kairos into BCM's existing provisioning pipeline. BCM already knows how to PXE boot nodes, rsync software images, and manage node lifecycle. We create a new BCM software image (`kairos-installer`) whose only job is to boot once, download a pre-built Kairos raw disk image over HTTP from BCM, and `dd` it onto the node's disk. After that, the node powers off and subsequently boots Kairos directly from its own disk via UEFI.

Why This Is Better

1. **Leverages existing infrastructure** — builds on your existing BCM DHCP, TFTP, NFS, and rsync pipeline. No new provisioning servers, no new networks.
2. **Additive, not destructive** — all changes to BCM are additions: a new software image, a new category, a target-device reconfiguration. Existing images, categories, and nodes are untouched. Cluster-wide defaults are not flipped.
3. **Reversible** — change a node's category back in `cmsh` and it reverts to standard HPC provisioning on next PXE boot.
4. **Repeatable** — the Kairos image is built once and deployed to any number of nodes.
5. **Immutable** — Kairos boots from a read-only ext2 loop-mount of `COS_ACTIVE` , immune to configuration drift.
6. **Idempotent re-deploys** — a dedicated pre-registration hook on the node deletes stale Palette edge-host records and auto-mints a fresh registration token when needed, so re-imaging a previously-provisioned node doesn't get stuck in Palette's "UID already registered" state.

Prerequisites

What We Need From You

Requirement	Detail
BCM head node	Running BCM 11.0. Accessible via SSH with root credentials — either directly from the build host or through an SSH jump host (ProxyCommand supported).
BCM services	<code>cmd</code> , <code>dhcpd</code> , <code>named</code> , <code>nfs-server</code> all active. Standard BCM provisioning must already be working for at least one existing category.
Network	The provisioning network where compute nodes live. BCM must be able to reach the Palette endpoint (whether on-prem or public SaaS).
Compute node(s)	Bare-metal server(s) registered in <code>cmsh</code> with the correct MAC and assigned to a category (any existing category is fine — we move the target device into the <code>kairos</code> category automatically). Each node must be configured to PXE boot.

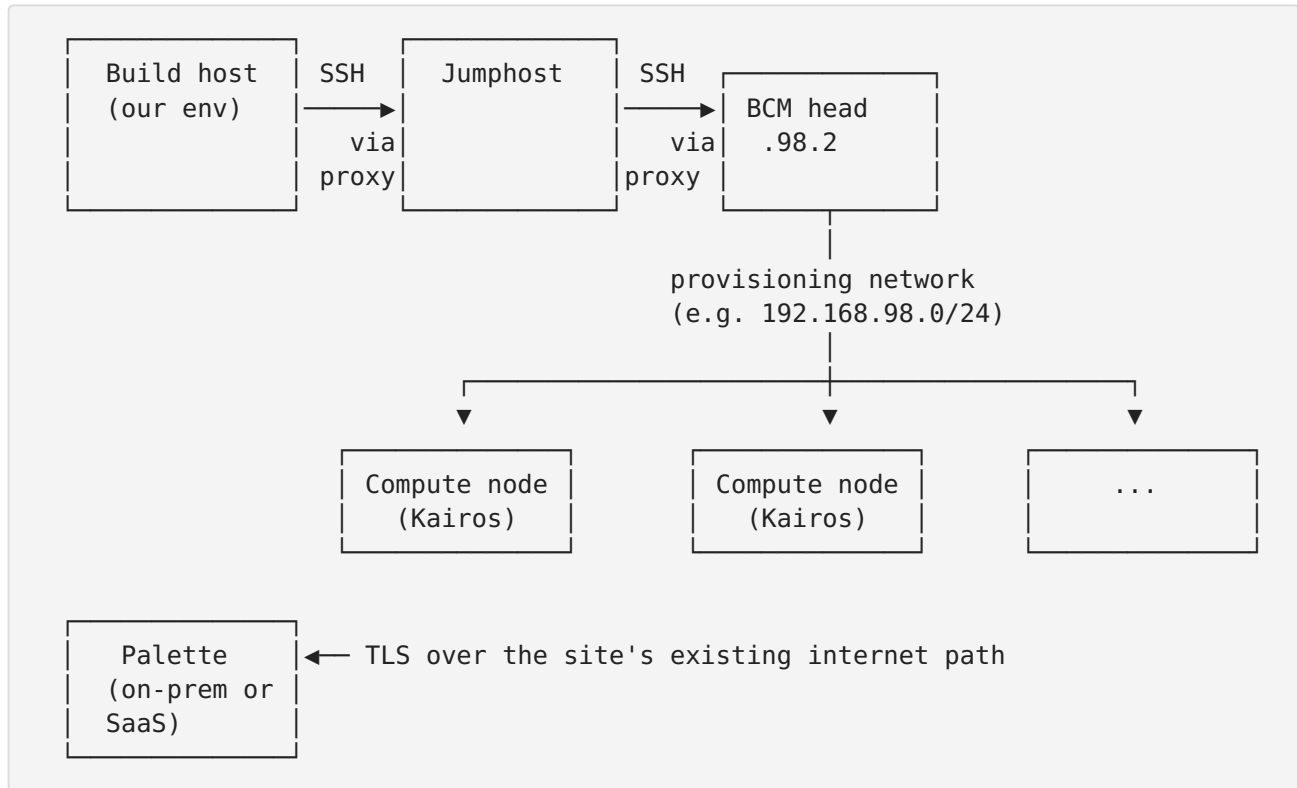
Requirement	Detail
Palette credentials	API endpoint, project name and UID, admin API key (with <code>edgeToken.create</code> + <code>edgehost.delete</code> permissions), and the CA certificate PEM if your Palette instance uses a self-signed cert. A pre-minted edge-host registration token is <i>optional</i> — if absent the node mints one on first boot using the admin key.
Out-of-band power management	IPMI / iDRAC / Redfish reachable for the target nodes. Used to force one-time PXE boot and power-cycle during the re-image.

What We Provide

Component	Detail
Kairos raw disk image	80 GB sparse raw image (~5.4 GB after lz4 compression), pre-built with Kairos OS + K3s + Palette <code>stylus-agent</code>
install-kairos.sh	Runs inside BCM's node-installer environment: streams the compressed image via <code>curl lz4 -d dd</code> onto the target disk, fixes the GPT backup header with <code>sgdisk -e</code> , registers a UEFI boot entry with <code>efibootmgr --create</code> , and powers off
palette-cleanup-stale.sh	Runs on the node as a stylus-agent ExecStartPre hook. Handles re-deploy edge cases: detects registration mode, deletes any stale Palette edge-host record for this SMBIOS UID via admin API, and auto-generates a fresh <code>edgeHostToken</code> if one isn't baked into the image
Ansible automation	<code>make discover</code> / <code>make kairos-build</code> / <code>make deploy-dd</code> / <code>make validate</code> targets
Validation suite	41-point automated validation covering BCM services, networking, Kairos OS, and Palette registration

Architecture

Network Topology (remote BCM, single provisioning subnet)



What Changes in BCM

We SSH to the BCM head node (directly or via configured jumphost) and make the following changes via `cmsh` and the filesystem:

1. **Software image `kairos-installer`** — cloned from `bcm_source_category`'s image (on a fresh BCM this is `default-image`; on an existing site we clone from the current operating category's image). Our `install-kairos.sh`, `kairos-install.service`, and `lz4` binary are injected; `eth0 / ens3` DHCP is added to `/etc/network/interfaces`; the PXE ramdisk is regenerated.
2. **Category `kairos`** — cloned from your existing category (e.g., "S-AI Partner Lab"). Set to `installmode=FULL` with `softwareimage=kairos-installer` and kernel parameters `console=ttyS0,115200n8 net.ifnames=0 biosdevname=0`. The `fsmounts` entries for `/cm/shared` and `/home` are removed (Kairos uses `COS_PERSISTENT` bind-mounts, not NFS). Three `nodeexecutionfilters` are installed to **exclude** the kairos category from BCM's `mounts`, `interfaces`, and `ntp` health checks — Kairos' immutable-OS architecture doesn't match what those checks expect, and without the filters the category would permanently show "health check failed" despite being functionally healthy.

3. **Target device reconfiguration** — the compute node is assumed to already be registered in cmsh. `bcm_target_node` in the inventory names the existing device. We set `category=kairos`, `softwareimage=kairos-installer`, and `installmode=FULL` on it. No new device records are created. (On a fresh local-KVM BCM the pipeline can alternatively register a `node001` from scratch.)
4. **IP forwarding + NAT** — `net.ipv4.ip_forward=1` and an iptables MASQUERADE rule on BCM's default-route interface (auto-detected — handles `eth1`, `ens*`, `enp*` alike). Only takes effect when compute nodes actually route through BCM for internet access; on sites where compute nodes have their own upstream gateway this is a harmless no-op.
5. **HTTP server** — systemd unit `kairos-http.service` runs a Python HTTP server on port `8888` from `/cm/shared/kairos/`. This is how compute nodes pull the compressed Kairos image during install.
6. **Kairos image upload** — the compressed `disk.raw.lz4` (~5.4 GB) is `scp`'d to `/cm/shared/kairos/` on BCM. Subsequent runs check size and skip re-upload when unchanged.

Not changed on a remote BCM (gated off by default):

- **DNS forwarders** — `bcm_manage_dns: false` by default; your site's existing DNS config is untouched. Only a fresh local-KVM BCM has its cluster-wide `nameservers` rewritten.
 - **Cluster defaults** — `bcm_manage_cluster_defaults: false` by default; `defaultcategory` and `nodebasename` on the `base` partition are not touched.
-

Per-Node Provisioning Flow

The node PXE boots exactly as it would for any BCM-managed node. The only difference is the software image BCM serves it.

Phase	Time	What Happens
POST + PXE	~3 min	UEFI POST, NIC sends DHCP DISCOVER, BCM responds with the registered IP + TFTP next-server, node fetches kernel + ramdisk from <code>/cm/images/kairos-installer/boot/</code>
BCM provisioning	~1 min	BCM's node-installer rsyncs <code>/cm/images/kairos-installer/</code> onto the target disk, installs GRUB-EFI into the ESP, <code>switchroot</code> s into the newly-synced filesystem (no reboot)
Kairos dd install	~2 min	<code>kairos-install.service</code> fires: stages binaries to <code>/dev/shm</code> , wipes signatures on non-boot disks (<code>wipefs -a</code>), streams <code>curl → lz4 → dd</code> onto the pinned target disk with <code>oflag=direct</code> , runs <code>sgdisk -e</code> to relocate the GPT backup header, runs <code>partprobe</code> to re-read partitions
UEFI boot entry	~5 s	<code>efibootmgr --create --disk <target> --part 1 --label Kairos --loader '\EFI\BOOT\bootx64.efi'</code> writes a new UEFI boot entry at the front of BootOrder — ensures the freshly-dd'd disk is picked on next power-up
Poweroff	~2 s	SysRq-triggered poweroff from RAM (the disk has been fully rewritten; <code>reboot</code> would fail)
Power on (manual or automated)	~3 min	Operator (or our iDRAC/Redfish hook) powers the server back on. UEFI picks the <code>Kairos</code> boot entry, loads GRUB-EFI from the new ESP, boots the active Kairos image
Cloud-config + registration	~2 min	Stylus-agent's ExecStartPre hooks run: <code>palette-cleanup-stale.sh</code> clears any stale edge-host record and mints a token if needed; <code>bcm-sync-userdata.sh</code> syncs hostname to Palette. Stylus registers the edge host with Palette, caches the hubble JWT on <code>COS_PERSISTENT</code>
Total	~13 min	From first power-on to <code>[UP]</code> in BCM + <code>healthy</code> in Palette

Quick Start (Remote BCM via SSH Jumphost)

The minimum configuration for a remote-BCM deploy through an SSH jumphost. Fuller detail on each step is in **Implementation Steps**.

Required inputs

Copy `inventory/group_vars/all.example.yml` to `inventory/group_vars/all.yml` and fill in at least the following. All other variables in the example file can stay at their defaults.

Variable	What it is	Example
<code>bcm_password</code>	BCM root SSH password	<code>"..."</code>
<code>bcm_ssh_host</code>	BCM's address on the provisioning network (reached via the jumphost)	<code>"192.168.98.2"</code>
<code>bcm_ssh_port</code>	SSH port on BCM	<code>22</code>
<code>bcm_ssh_proxy_jump</code>	Jumphost in <code>user@host</code> form — applied as an OpenSSH <code>ProxyCommand</code> to every BCM call	<code>"partner@100.90.255.42"</code>
<code>bcm_ssh_proxy_key</code>	Private key for jumphost auth (<code>~</code> is expanded)	<code>"~/ .ssh/sai"</code>
<code>bcm_internal_ip</code>	BCM's IP on the provisioning network (baked into the compute node's HTTP URL)	<code>"192.168.98.2"</code>
<code>bcm_internal_cidr</code>	CIDR for NFS exports and DHCP pool scope	<code>"192.168.98.0/24"</code>
<code>bcm_manage_dns</code>	Must stay false on a customer BCM — skips the cluster-wide DNS rewrite	<code>false</code>
<code>bcm_manage_cluster_defaults</code>	Must stay false on a customer BCM — skips flipping <code>defaultcategory / nodebasename</code>	<code>false</code>
<code>bcm_target_node</code>	Existing cmsh device name to re-image	<code>"edge-4c4c454400485610804bc3c04f4e4434"</code>

Variable	What it is	Example
<code>kairos_target_mac</code>	Optional. MAC of the compute node's provisioning NIC. When set, <code>deploy-dd</code> does <code>set mac <value></code> on <code>bcm_target_node</code> — mirrors the local-KVM path so the MAC mapping is owned by this deploy rather than pre-configured in <code>cmsh</code> . Leave empty to keep the existing MAC	<code>"4c:4c:45:44:00:48"</code>
<code>kairos_target_ip</code>	Optional. IP to assign on the provisioning network. When set, <code>deploy-dd</code> does <code>set ip <value></code> on <code>bcm_target_node</code> . Must be inside <code>bcm_internal_cidr</code> and outside the DHCP pool (<code>.16 – .250</code>). Leave empty to keep the existing IP	<code>"192.168.98.10"</code>
<code>kairos_profile</code>	Default default-kairos. Namespaces the build artifact and BCM-side state (<code><profile>-installer</code> software image, <code><profile></code> category, <code>/cm/shared/kairos/<profile>/</code> upload). Multiple profiles coexist on the same BCM — pick a different name to keep multiple builds side-by-side	<code>"gpu-kairos"</code>

Variable	What it is	Example
<code>kairos_canvos_args</code>	Optional dict. Overrides any subset of CanvOS <code>.arg</code> keys (<code>OS_VERSION</code> , <code>UPDATE_KERNEL</code> , <code>CIS_HARDENING</code> , <code>UBUNTU_PRO_KEY</code> , <code>IMAGE_REGISTRY</code> , <code>CUSTOM_TAG</code> , ...). Defaults are in <code>roles/kairos_build/defaults/main.yml</code> ; arbitrary new keys flow through to <code>.arg</code> verbatim	<code>{OS_VERSION: "22.04", UPDATE_KERNEL: "true"}</code>
<code>kairos_extra_apt_packages</code>	Optional list. Extra apt packages installed in the Kairos image via a Dockerfile RUN block. Empty list = no extras	<code>[htop, tcpdump, jq]</code>
<code>kairos_user_data</code>	Optional raw YAML. Layered into <code>/oem/99_userdata.yaml</code> on top of the BCM/Palette base config. Empty = no extra userdata file written	(see worked example)
<code>bcm_source_category</code>	Existing BCM category to clone for the new <code>kairos</code> category (inherits disksetup + mon templates + FinalizeXML)	<code>"S-AI Partner Lab"</code>
<code>kairos_target_disk</code>	Disk device on the compute node that <code>dd</code> will overwrite	<code>"/dev/nvme0n1"</code>
<code>kairos_wipe_disks</code>	Sibling disks to <code>wipefs -a -f</code> before <code>dd</code> (clears stale LVM/DRBD signatures)	<code>"nvme1n1 nvme2n1 nvme3n1 nvme4n1"</code>
<code>palette_endpoint</code>	Palette API hostname	<code>"s-ai.lan"</code>
<code>palette_project_name</code>	Palette project	<code>"Default"</code>
<code>palette_project_uid</code>	Palette project UID	<code>"69a81eb..."</code>

Variable	What it is	Example
<code>palette_api_key</code>	Admin API key with <code>edgeToken.create</code> + <code>edgehost.delete</code>	<code>"..."</code>
<code>palette_ca_cert</code>	PEM block — only needed if Palette uses a self-signed CA	<code>-----BEGIN CERTIFICATE----- ...</code>
<code>palette_token</code>	Optional. Pre-minted edge-host registration token. If omitted, the node auto-mints one on first boot using <code>palette_api_key</code> + <code>palette_project_uid</code>	<code>"..."</code>

Sequence of commands

```
# 1. (optional) Discover the BCM – prompts for the same credentials and emits
#    a starter inventory with the network/category/node fields pre-filled.
make discover

# 2. Fill in inventory/group_vars/all.yml with values from above.
cp inventory/group_vars/all.example.yml inventory/group_vars/all.yml
$EDITOR inventory/group_vars/all.yml

# 3. Build the Kairos raw image once (runs entirely on the build host,
#    no BCM interaction). ~30 min.
make kairos-build

# 4. Upload the image to BCM and configure the kairos-installer + category +
#    target device. ~3–5 min on direct network; up to ~30 min on the first
#    run through a slow jumphost (SCP of the ~5.4 GB image is the bottleneck).
#    Subsequent runs skip the SCP when sizes match.
make deploy-dd

# 5. Power-cycle the target compute node to PXE boot (via iDRAC, IPMI, or
#    Redfish). BCM drops the kairos-installer ramdisk onto it; the installer
#    dd's the Kairos image onto the target disk, registers a UEFI boot entry,
#    and powers off. Power the node back on – UEFI boots Kairos from disk.

# 6. Validate end-to-end – SSHes to BCM through the jumphost, then from BCM to
#    the booted Kairos node. ~15 sec.
make validate
```

All BCM-facing commands in steps 4 and 6 route through `bcm_ssh_proxy_jump` via a per-run SSH config file — no manual tunnel setup is needed on the build host.

Implementation Steps

Step 0 (optional): Discover existing BCM state — `make discover`

What: Interactive playbook that SSHes to the BCM head node (through your jumphost), reads the internal network definition from cmsh, enumerates categories, software images, and registered devices, and writes a site-specific inventory starter file at `bcm-discovery-<bcm-hostname>.yml`.

When to use: Run first on any BCM you haven't deployed to before, to harvest the right values for `bcm_internal_ip`, `bcm_internal_cidr`, `bcm_external_dns`, and to see which category/image names already exist on the BCM so you can pick the right `bcm_source_category`.

Non-interactive invocation (so you can script it):

```
ansible-playbook playbooks/discover-bcm.yml \
  -e bcm_host=192.168.98.2 -e bcm_port=22 -e bcm_user=root \
  -e bcm_pass='<root-password>' \
  -e bcm_proxy_jump='user@jumphost.example.com' \
  -e bcm_proxy_key=~/.ssh/sai'
```

Step 1: Build the Kairos image — `make kairos-build`

What: Builds a bootable 80 GB raw disk image containing Kairos OS, K3s, the Palette stylus-agent, and our BCM integration scripts.

Where: On the build host (our CI/workstation). Does not touch BCM.

Process:

1. **CanvOS ISO build** (~10 min, Earthly + Docker) — clones Spectro Cloud's CanvOS submodule, renders the `.arg` file with the target registry/arch/kernel, copies our overlay files (BCM compatibility scripts, systemd drop-ins, the `palette-cleanup-stale.sh` hook), patches the Earthfile to include `ifupdown` + `nfs-common`, and runs `earthly +iso` to produce the Palette Edge Installer ISO.
2. **Raw disk build under OVMF/UEFI** (~8 min, headless QEMU) — generates a BCM- ↔ -Kairos ed25519 SSH keypair, renders the cloud-config with the Palette + BCM parameters, creates a 4 MB FAT32 user-data image, boots a headless QEMU VM *under OVMF UEFI firmware* (not SeaBIOS) so `kairos-agent install` produces a real EFI System Partition with `\EFI\B00T\bootx64.efi` — required for physical UEFI servers to boot the resulting raw.
3. **Post-processing** — `e2fsck` + `tune2fs -O metadata_csum` on each ext4 partition for GRUB compatibility, patches `bootargs.cfg` with `net.ifnames=0 biosdevname=0` for stable NIC naming, sets GRUB timeout to 5 s for unattended boot, sparse-trims the raw file with `fallocate --dig-holes` (80 GB virtual → ~8.7 GB on disk), writes a SHA-256 checksum.

Output: `build/kairos-disk.raw` (80 GB sparse, ~8.7 GB actual), `build/palette-edge-installer.iso` (~1.5 GB), `build/cloud-config.yaml` (~8 KB rendered), `build/bcm-kairos-key` + `.pub` (ed25519 keypair).

Time: ~25 min total (once; the same image deploys to all nodes).

Step 2: Deploy to BCM — `make deploy-dd`

What: Uploads the Kairos image to BCM, creates the installer software image, configures the `kairos` category, targets the specified compute device, and regenerates the PXE ramdisk.

Where: Runs on the build host, SSHes to BCM through the jumphost to make all changes.

What it does (7 steps):

1. **Compress + upload** — `lz4 -f build/kairos-disk.raw build/kairos-disk.raw.lz4` (80 GB → ~5.4 GB), `scp` to `root@<bcm>:/cm/shared/kairos/disk.raw.lz4` via the configured SSH config. Skip-if-size-matches check avoids re-uploads on subsequent runs.
2. **HTTP server** — creates `/etc/systemd/system/kairos-http.service` serving `/cm/shared/kairos/` on port 8888. Enables + starts.
3. **Create installer image** — `cmsh softwareimage; clone <bcm_source_category>'s image> kairos-installer; commit`. Reuses existing if already present. Waits for BCM to populate `/cm/images/kairos-installer/usr/`.
4. **Install dd service + lz4 into the image** — installs `lz4` on BCM via `apt` (does NOT touch BCM's `/etc/resolv.conf`), copies it + our `install-kairos.sh` into `/cm/images/kairos-installer/usr/local/{bin,sbin}/`, drops `kairos-install.service` into the image's `systemd` multi-user target. Also appends `eth0 / ens3` DHCP stanzas to the image's `/etc/network/interfaces`.
5. **Configure kairos category** — clones from `bcm_source_category`, sets `installmode/newnodeinstallmode/installbootrecord/kernelparameters`, strips `/cm/shared` and `/home` fsmounts, and installs three `nodeexecutionfilters` (`Exclude, category=kairos`) against the `mounts`, `interfaces`, and `ntp` monitoring setups.
6. **Target device** — when `bcm_target_node` is set: `cmsh device; use <target>; set category kairos; set softwareimage kairos-installer; set installmode FULL; commit`. On a fresh BCM without `bcm_target_node`, falls back to registering `node001` with an explicit IP + MAC.
7. **Regenerate ramdisk** — `cmsh softwareimage; use kairos-installer; createramdisk -w` rebuilds the PXE `initrd` for the `kairos-installer` image so TFTP can serve it. Takes 5–10 min.

Time: ~10 min on a fresh/local BCM, or ~30 min on a remote BCM (the SCP upload of the ~5.4 GB image through a jumphost is the bottleneck). Skip-if-exists on subsequent runs drops that to ~3 min.

Key inventory variables (see `inventory/group_vars/all.example.yml`):

Variable	Purpose
<code>bcm_password</code>	BCM root SSH password
<code>bcm_ssh_host</code> / <code>bcm_ssh_port</code>	BCM address (e.g., <code>192.168.98.2:22</code> on a remote, <code>localhost:10022</code> on local-KVM)
<code>bcm_ssh_proxy_jump</code> / <code>bcm_ssh_proxy_key</code>	Jumphost in <code>user@host</code> form + key path (empty when BCM is directly reachable)
<code>bcm_internal_ip</code> / <code>_netmask</code> / <code>_cidr</code>	BCM's provisioning-network identity (bake into <code>install-kairos.sh</code> 's HTTP URL)
<code>bcm_source_category</code>	Existing category to clone for <code>kairos</code> (e.g., "S-AI Partner Lab")
<code>bcm_target_node</code>	Name of the existing cmsh device to re-image
<code>bcm_manage_dns</code>	Default <code>false</code> on remote — leaves site DNS alone
<code>bcm_manage_cluster_defaults</code>	Default <code>false</code> on remote — no cluster-wide changes
<code>kairos_target_disk</code>	Disk device to dd onto (e.g., <code>/dev/nvme0n1</code>)
<code>kairos_wipe_disks</code>	Sibling disks to <code>wipefs -a</code> before dd (clears stale LVM/DRBD signatures)
<code>palette_endpoint</code> / <code>palette_project_name</code> / <code>palette_project_uid</code>	Palette coordinates
<code>palette_api_key</code>	Admin key (baked to node for cleanup + token auto-gen)
<code>palette_ca_cert</code>	Self-signed CA PEM for non-public Palette
<code>palette_token</code>	<i>Optional</i> pre-minted edge-host token; auto-generated when absent

Step 3: PXE boot the compute node

What: Force a one-time PXE boot via iDRAC/Redfish, then cycle power once more after the `dd` completes.

Automatable via racadm (Dell):

```
# Before cycle – set one-time PXE boot
racadm set iDRAC.ServerBoot.BootOnce Enabled
racadm set iDRAC.ServerBoot.FirstBootDevice PXE
racadm serveraction powerdown
# ...wait until powerstatus=OFF...
racadm serveraction powerup

# Watch: BCM DHCP + TFTP → node-installer → install-kairos.sh → dd → UEFI boot
entry → SysRq poweroff
# ...wait until powerstatus=OFF again (dd done)...

# After dd – power back on
racadm serveraction powerup
# UEFI now has the `Kairos` entry at the front of BootOrder and boots Kairos from
disk
```

The **first** PXE boot exists because the current BootOrder doesn't contain a Kairos entry yet — UEFI has to fall through to PXE. The **second** power-on boots Kairos from the disk directly (no PXE) because `install-kairos.sh` registered the Kairos boot entry. Subsequent reboots stay on disk.

Step 4: Validate — `make validate`

What: A bash script rendered from an Ansible template. Uses the same SSH chain as `deploy-dd` (through the jump host) to run 41 checks across BCM and the compute node.

BCM checks (18): SSH reachability, service status (`cmd`, `dhcpd`, `named`, `nfs-server`, `rsyncd`, `HTTP:8888`), internal IP present on any interface, external interface detection, IP forwarding, external DNS resolution, internet reachability, cluster state (head node UP, target node registered with correct IP + `kairos` category), `kairos-installer` image present, raw disk on `/cm/shared/`.

Kairos node checks (23): SSH reachability (via BCM jump host), ping, OS version (Ubuntu 22.04 base), Kairos release + version, kairos-agent version, kernel version, IP address, gateway, DNS resolver, external DNS resolution, internet access, stylus-agent active, Palette registration logged, boot cmdline has `net.ifnames=0` + Kairos boot chain markers, COS partitions present (OEM/RECOVERY/STATE/PERSISTENT), root immutable, disk free, `/oem/*.yaml` present.

Sample output:

```
=====
E2E Deployment Validation
=====
== BCM Head Node (port 22) ==
[PASS] BCM SSH – port 22
[PASS] cmd service
[PASS] dhcpd
[PASS] named (DNS)
[PASS] nfs-server
[PASS] rsyncd (873)
[PASS] HTTP server (8888) – Kairos image server
[PASS] BCM internal IP – 192.168.98.2 on enp6s0
[PASS] BCM external iface – enp6s0
[PASS] IP forwarding
[PASS] External DNS
[PASS] Internet access – HTTP 301
[PASS] Head node (cmsh)
[PASS] <target-node> registered
[PASS] <target-node> IP – <ip>
[PASS] <target-node> category – kairos
[PASS] kairos-installer image
[PASS] Kairos raw image – /cm/shared/kairos/disk.raw.lz4

== Kairos Compute Node ==
[PASS] Kairos SSH
[PASS] Kairos ping
[PASS] OS – Ubuntu 22.04.5 LTS
[PASS] Kairos release – v4.0.3
[PASS] kairos-agent – v2.27.0
[PASS] Kernel – 6.8.0-106-generic
[PASS] IP address
[PASS] Gateway
[PASS] DNS resolver
[PASS] External DNS resolution
[PASS] Internet access – HTTP 301
[PASS] stylus-agent
[PASS] Palette registration
[PASS] net.ifnames=0
[PASS] Kairos boot chain
[PASS] COS_OEM
[PASS] COS_RECOVERY
[PASS] COS_STATE
[PASS] COS_PERSISTENT
[PASS] Root immutable
[PASS] Disk free
[PASS] OEM config

PASS: 41/41   WARN: 0/41   FAIL: 0/41
```

Scaling to Multiple Nodes

Once the initial setup on BCM is done (Steps 1–2 complete, image uploaded, category + filters + installer configured), adding more nodes is straightforward:

1. Your BCM admin adds each new node to cmsh (any standard BCM onboarding process — existing site tooling applies).
2. Update `bcm_target_node` in `inventory/group_vars/all.yml` to the new device's cmsh name.
3. Re-run `make deploy-dd` — `skip-if-exists` means the image isn't re-uploaded; only the cmsh reconfiguration runs (~1 min).
4. PXE-boot-once the new node.

Or, for fleet provisioning: loop over target-node names, automating the per-node iDRAC cycle with a small shell script using the same racadm pattern.

One Kairos raw image + one admin API key provisions any number of nodes. No per-node Palette UI interaction is required.

Re-deploying a Node (Idempotent Re-image)

A previously-provisioned Kairos node can be re-imaged safely:

- Its SMBIOS UUID is deterministic, so Palette would normally reject a re-registration with *"Edge host already registered"*. Our on-node `palette-cleanup-stale.sh` handles this: before `stylus-agent` starts, it checks for a stale record for this UID and deletes it via the admin API.
- Similarly, any orphan DRBD/Linstor/LVM signatures on sibling data disks from a prior cluster get wiped by `install-kairos.sh`'s `wipefs` pass, preventing Kairos (or a later cluster profile) from auto-activating stale volumes.
- The UEFI boot entry is rewritten: `install-kairos.sh` removes any existing Kairos entries before creating the fresh one, so re-imaging doesn't stack duplicate entries in `BootOrder`.

The re-deploy procedure is identical to the first-time procedure: `make deploy-dd` (if the image or config changed; otherwise skip), then PXE-boot-once via iDRAC.

Rollback

To return a node to standard BCM management:

```
# 1. In cmsh, revert the device's category and clear the Kairos softwareimage
override
cmsh -c "device; use <node>; set category <original-category>; clear
softwareimage; set installmode AUTO; commit"

# 2. Delete the edge host record in Palette (UI → Edge Hosts → Delete, or
DELETE /v1/edgehosts/<uid> via admin API)
#    to release the UID before the next re-registration

# 3. Power cycle the node. BCM now provisions the original HPC image (via PXE)
instead of kairos-installer.

# 4. (Optional) Remove the kairos category's nodeexecutionfilters if you want the
mounts/interfaces/ntp
#    healthchecks re-enabled for any remaining Kairos nodes
```

Total time: a few cmsh commands plus one PXE boot.

Risk Mitigation

Risk	Mitigation
<code>dd</code> overwrites the wrong disk	<code>kairos_target_disk</code> is pinned in inventory (e.g., <code>/dev/nvme0n1</code>). The script never auto-detects on remote deploys. Sibling disks are handled by an explicit <code>kairos_wipe_disks</code> list.
BCM provisioning fails	Node stays on the BCM node-installer / NFS root — no data loss. Power-cycle to retry.
Kairos image corrupt	<code>make validate</code> catches it — OS, partition, service, and registration checks all flag mismatches. Rebuild the image and re-run <code>deploy-dd</code> .
Palette registration fails with "UID already registered"	<code>palette-cleanup-stale.sh</code> deletes the stale record before stylus-agent attempts registration. Requires a valid <code>palette_api_key</code> in inventory.
<code>edgeHostToken</code> is empty or stale	If no token is baked in, <code>palette-cleanup-stale.sh</code> mints a fresh one via the admin API (30-day expiry) and injects it into <code>/oem/90_custom.yaml</code> before stylus-agent starts.
Node doesn't PXE boot	Verify BIOS boot order, check BCM DHCP logs (<code>journalctl -u dhcpcd</code>), confirm MAC matches cmsh. The one-time PXE override via racadm bypasses persistent BootOrder changes.

Risk	Mitigation
BCM healthchecks flag the node	Expected for Kairos' architecture (different mount/interface semantics). Our deploy automatically installs <code>nodeexecutionfilters</code> to exclude the <code>kairos</code> category from those specific checks, keeping <code>cmsh device list</code> clean.
Disk space on BCM	<code>/cm/shared/kairos/disk.raw.lz4</code> is ~5.4 GB. The <code>kairos-installer</code> software image in <code>/cm/images/</code> is ~2 GB. Allow ~10 GB free on BCM.

Deliverables

1. **Ansible automation** — `make discover`, `make kairos-build`, `make deploy-dd`, `make validate` targets, all driven from a single inventory file (`inventory/group_vars/all.yml`; example template at `inventory/group_vars/all.example.yml`).
2. **Kairos raw disk image** — pre-built, tested, UEFI-bootable with Palette stylus-agent and our BCM integration cloud-config baked in.
3. **BCM configuration artifacts** — `kairos-installer` software image, `kairos` category, category-scoped healthcheck exclusion filters, target device reconfiguration, HTTP image server systemd unit.
4. **palette-cleanup-stale.sh** — on-node pre-registration hook covering all re-deploy edge cases (stale Palette records, missing tokens, re-registration).
5. **UEFI support** — image built with OVMF firmware, `install-kairos.sh` writes a UEFI boot entry via `efibootmgr --create` so physical UEFI servers boot the freshly-dd'd disk without any one-time-boot override.
6. **Discovery tool** — `make discover` / `playbooks/discover-bcm.yml` captures an existing BCM's network, category, image, and device state into a site-specific starter inventory file.
7. **Validation suite** — 41-point automated check covering BCM services, networking, Kairos OS, partitions, services, and Palette registration. Works locally (against a KVM BCM) and remotely (through an SSH jump host).
8. **Documentation** — this POC document, an `all.example.yml` inventory template, the `docs/pipeline-deep-dive.md` architecture reference.

Confidential — Kairos Edge Deployment POC